ARCHITECTURE DOCUMENT

# Private AI Connector Workspace

Multi-firm cloud index with a separate local index, Google Drive and Microsoft 365 connectors, and cited AI answers.

November 3, 2026

Status (November 2026): the cloud service, the Google Drive and Microsoft 365 connectors, the file reader, and the desktop app's local-plus-cloud search are built and tested on a development machine (sections 13 and 15 to 21). Nothing has been deployed to AWS. Nothing here has been priced with the AWS calculator or reviewed by a lawyer or auditor. Where a section describes something not yet built, it says so.

# Contents

1. Decisions already made
2. Overview
3. Components
4. Tenant isolation rules (each one gets an automated test)
5. Permission-aware search
6. Search and answer quality
7. Connector framework
8. Detection and audit
9. Threat table
10. Build order
11. Cost (ballpark only, unverified; use the AWS Pricing Calculator)
12. Open decisions
13. Built so far
14. Solo users
15. Desktop app: local and cloud together
16. Managing connectors from the desktop app
17. How search works as built
18. Links to the original files
19. When connectors sync
20. Reliability findings from testing
21. Running cost per user (estimate)
22. Known gaps
23. Test status
24. Choosing the model and limiting use
25. Reading scans and pictures (OCR)
26. Search by meaning and reranking
27. Company knowledge (first phase)
28. Shared AI agents (first slice, read-only)
29. Keeping and deleting agent work

30. Dropbox and DocuSign connectors

31. Who a connected source is for: only me, or the entire workspace

32. Emails and meetings prepared by agents (no mailbox or calendar access)

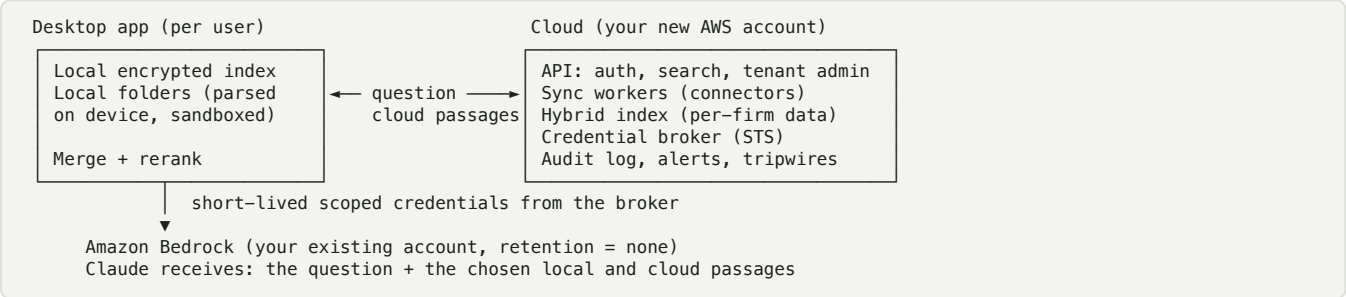
33. Sending a prepared email from the app (Outlook)

Figures: architecture, data flow, data inventory

# 1. Decisions already made

| Decision | Choice |
|---|---|
| Who uses it | Several firms (multi-tenant product) |
| Priority | Speed, answer accuracy and output quality over maximum privacy |
| Cloud index | One unified, searchable index across connectors (Glean-style) |
| Local files | Stay on the device in their own encrypted index. Never uploaded to the cloud index. The AI can still read matching local passages at question time |
| Isolation | Shared servers, fully separate data per firm. Upgrade path: dedicated AWS account for a firm that asks |
| Model | Claude Sonnet 5.5 on Amazon Bedrock with zero data retention (`none`) required before any request |
| First connectors | Microsoft 365 first, Google Drive in parallel once Google verification clears |
| Enclaves | Not affordable now. Revisit when revenue allows |

# 2. Overview




Key property: **the cloud never receives local text.** The app asks the cloud for matching cloud passages, searches locally, merges, and calls Bedrock itself. The cloud has no channel into the device.

# 3. Components

**Control plane (shared, small):** firms, users, roles, billing, connector settings. Sign-in through Amazon Cognito with Microsoft and Google federation. Every access token carries the firm ID.

**Data plane (shared servers, separate data per firm):**

- Postgres (RDS) with pgvector. Shared tables with a firm ID on every row and **forced row-level security as the primary lock**: the database itself hides every row that doesn't belong to the firm the transaction was opened for. The firm is set by the server from the verified token, never taken from the client, and it lasts only for that transaction.
- S3 for original and extracted files, one prefix per firm, encrypted with a **per-firm KMS key**. Bucket policy denies any access whose encryption context names another firm.
- Secrets Manager for connector refresh tokens, one secret per firm and connector, readable only by that firm's role.

**Sync workers (ECS Fargate):** run connectors, parse files, chunk, embed, write to the firm's schema. They hold no inbound ports. Parsing runs in a sandbox with no network.

**Search API:** takes a question from an authenticated user, applies the firm filter and that user's permission filter, runs hybrid search, returns passages.

**Credential broker:** Cognito identity pool that issues short-lived credentials allowing only `bedrock:InvokeModel` on the approved model. Tagged per firm for usage metering and limits.

**Desktop app:** keeps local indexing as it is today. Adds sign-in, a cloud search client, merging, and the Bedrock call with broker credentials.

## 4. Tenant isolation rules (each one gets an automated test)

1. The firm ID comes only from the verified token, never from request fields.
2. Every table has a firm ID, and row-level security rejects rows from other firms even if a query forgets the filter.
3. Each firm has its own KMS key. Key policy requires the firm ID in the encryption context.
4. Database roles are per job, not per firm: `app_api` (read-only search), `app_worker` (writes synced data) and `app_provisioner` (creates and deletes firms, sees no firm data). None can log in directly, bypass row security, or own a table. An earlier draft had a role per firm; it added operational weight without stopping anything row-level security doesn't already stop.
5. S3 and Secrets Manager access is limited by firm tags on the role session.
6. Logs never contain document text, queries or tokens. Only IDs and counts.
7. A cross-firm test suite runs in CI: sign in as firm A, try every endpoint for firm B's data, expect denial.
8. Offboarding a firm deletes its schema, its S3 prefix and its secrets, then schedules its key for deletion.

**Honest limit:** these rules stop bugs and misconfiguration from leaking one firm's data to another. They do not stop an attacker who takes over the running service itself, because the service can reach every firm. Keep that service small: no shell access, immutable containers, least-privilege roles, no standing human access, and alerts on unusual cross-firm volume.

## 5. Permission-aware search

- Connectors read each file's sharing settings (Microsoft Graph permissions, Google Drive ACLs) and store them as "who may read this".
- Every search filters by the signed-in user's identity and groups, so a user never gets a file they couldn't open in the source system.
- Permissions are re-synced on a schedule and on change notifications. A file whose permissions can't be read is hidden, not shown.
- Firm admins can add **walls** (matter or client restrictions) on top of source permissions.

## 6. Search and answer quality

1. **Hybrid retrieval:** Postgres full-text (keyword, exact names) plus pgvector (meaning), combined by reciprocal rank fusion. The existing local search already has name and entity handling, and the cloud index reuses the same rules (joined words, entity aliases).
2. **Rerank:** a reranking model re-sorts the top ~50 passages. For the merged cloud and local list, reranking runs on the device through Bedrock so local text still never goes to the cloud. Check the reranker's data-retention terms before enabling it.
3. **Structure-aware chunking:** by heading, page and table, so citations point to a page or cell.
4. **Whole-document summaries:** as today, a summary question sends the full text of the top document.
5. **Citations:** every answer lists its sources and whether each was local or cloud.
6. **Evaluation set:** 50–100 real questions with known answers per sector, run before each release. Without this, "accurate" is a guess.

## 7. Connector framework

Each connector implements: `authorize()`, `list(changes_since)`, `fetch(file)`, `permissions(file)`, `revoke()`. Scopes are read-only. Each declares what data it touches, so the admin screen can show it. Delta sync, not full re-crawls. Microsoft 365 uses Graph delta queries and change notifications; Google Drive uses the Changes API.

**Gates:**

- Microsoft: multi-tenant app registration, publisher verification, and each firm's admin consents once.
- Google: reading all of a customer's Drive files is a restricted scope. As I understand it, this requires app verification and an annual third-party security assessment. Confirm in Google's current documentation and start early. A narrower scope (user picks files) avoids this but is a worse product.

## 8. Detection and audit

- Immutable audit log of sign-ins, searches (metadata only), connector changes and admin actions, exported to the firm on request.
- CloudTrail, GuardDuty and alarms for unusual volume, new regions and key-policy changes.
- **Tripwires:** a few decoy documents per firm, hidden from normal search ranking. Any read raises an alert. The `Honeytoken` files also reveal if a connector account is being abused.
- No standing access for staff. Break-glass access requires a second person and is logged and reviewed.

## 9. Threat table

| Event | What an attacker gets | Mitigation |
|---|---|---|
| Bug leaks across firms | Another firm's passages | Rules 1–7; CI cross-firm tests |
| Cloud index corrupted or poisoned | Wrong or hostile cloud passages | Prompt treats source text as data; sources always shown; local index unaffected |
| Cloud service taken over | Cloud index for all firms | Small attack surface; alerts; cannot reach devices or local files |
| Stolen connector token | One firm's read-only connector access | Per-firm secrets; revoke on alert |
| Laptop stolen and signed in | That user's local files and cloud permissions | Disk encryption, short sessions, remote sign-out, device list |
| Insider with admin access | Anything the role allows | No standing access; two-person break-glass; audit |

## 10. Build order

1. **Foundation:** new AWS organization and account, infrastructure as code, Cognito, per-firm provisioning, the cross-firm test suite, audit logging.
2. **Microsoft 365 connector:** sync, permissions, hybrid index, search API.
3. **Desktop client:** sign-in, cloud search, merge and rerank, credential broker, Bedrock call. Receipt shows local vs cloud sources.
4. **Evaluation set and tuning**, then a pilot with one friendly firm using test data.
5. **Google Drive** after verification, tripwires and usage limits, admin screens.

6. **Compliance track in parallel:** SOC 2 Type II preparation, penetration test, data processing agreement, terms of service, incident-response plan.

## 11. Cost (ballpark only, unverified; use the AWS Pricing Calculator)

| Item | Rough monthly |
|---|---|
| Fixed baseline (small RDS, containers, network, logging, monitoring) | a few hundred dollars |
| Per firm (KMS key, storage, sync compute, secrets) | tens of dollars at small scale |
| Claude answers (Sonnet 5.5: $2 / $10 per million input / output tokens) | a few cents per answer, more for whole-document summaries |
| Embeddings and rerank | small per document and per query |

One-time or yearly costs that are easy to underestimate: SOC 2 tooling and audit, a penetration test, Google's security assessment, cyber insurance, and legal review of your terms and data processing agreement. These can exceed the cloud bill for a long time.

## 12. Open decisions

- Name of the company and product (affects domains, app registrations and legal documents).
- Whether firms may later sync some local folders to the cloud index (not planned now).
- Whether a firm needs its own dedicated AWS account at launch.
- Who is the first pilot firm, and what are its engagement-letter and consent requirements.

## 13. Built so far

`backend/` is a Rust crate (`workspace-cloud`) with:

- `migrations/0001_tenancy.sql`: tables, composite foreign keys that stop a row from pointing at another firm's parent, forced row-level security, per-job roles, and an append-only audit table.
- `src/auth.rs`: token verification. RS256 only; checks signature, issuer, audience and expiry; refuses tokens with no valid firm ID, `none` and HMAC-forged tokens.
- `src/db.rs`: firm-scoped transactions, permission-aware search (files with unknown readers are hidden from everyone), audit writes, firm provisioning and offboarding.
- `tests/isolation.rs`: 11 cross-firm tests against a real Postgres, plus 5 token tests. Run `backend/scripts/test.sh`. Each of five deliberate weakenings of the rules (open policy, plain foreign key, row security off, audit UPDATE granted, unknown-permission files visible) made at least one test fail.
- `src/api.rs`, `src/jwks.rs`, `src/server.rs`, `src/main.rs`: the HTTP API (`/v1/me`, `/v1/search`, health checks), ID-token sign-in with the identity provider's rotating keys, a database pool that requires TLS, and a `migrate` command. Request size, time and concurrency are capped. Firms that were offboarded or suspended are refused even with a still-valid token. 7 more tests cover this over real HTTP, including requests from two firms sharing one database connection.
- Connectors: `src/connectors/` (Microsoft Graph and Google Drive, with the translation of each provider's sharing settings into who may read a file), `src/sync.rs` (resumable sync, periodic full listing, mark-and-sweep), `src/queue.rs` and `src/worker.rs` (leased work queue and the worker loop), `src/oauth.rs` (consent flow with signed state), `src/secrets.rs` (per-firm secret names) and the `/v1/connections` and

`/oauth/callback` routes. 36 more tests run against stand-in provider servers, including connect, sync, search and disconnect end to end. See `backend/README.md` for the known limits.

- `backend/scripts/smoke.sh` starts the real server against a throwaway database and checks it with curl.
- CI runs the same tests against a Postgres service.

`infra/` holds the AWS infrastructure as OpenTofu code: network, encryption keys, storage, database, sign-in, audit logging, alerts, and the tag-based roles that confine each session to one firm's files, keys and secrets. It passes `tofu validate` only; it has never been planned or deployed (see `infra/README.md`).

- File reading: `src/extract.rs` (PDF with a text layer, Word, PowerPoint, Excel; labelled by page, slide or sheet; bounded and hostile-file safe), `src/extract_process.rs` and `src/bin/workspace-extract.rs` (the restricted reader program and the code that runs it), and the sync changes that read each downloaded file, store passage labels, and read old files again when the reader improves. About 25 more tests, including real runs of the reader program with hung, flooding, crashing and damaged cases.

Not built yet: OCR for scanned pages and photos, old .doc/.ppt files, network and file-system isolation for the reader (deployment must provide it), the AWS Secrets Manager store for connector tokens (nothing can be saved outside development without it), group-membership resolution, per-user rate limits, a container image and the services that run it, database login through IAM, embeddings and reranking, the Bedrock credential broker, and the desktop client changes. Vector search needs the pgvector extension, which isn't installed on this Mac; RDS and Aurora provide it.

## 14. Solo users

Personal Microsoft accounts are supported alongside work accounts, so a single person can use the product. A solo user is a firm of one: same isolation, one person, one connection or more. What is not built for them yet: self-service sign-up (Cognito currently only lets us create users, and a new firm must be created with its first user), a free or low-cost plan, and billing. These are product decisions as much as engineering ones.

## 15. Desktop app: local and cloud together

Built in `src-tauri/`:

- `cloud.rs`: the cloud client. Sign-in through Cognito's hosted page with PKCE and a one-shot callback listener on this machine; tokens kept in the OS credential store (a private file in development builds); one refresh and retry on an expired token; HTTPS required (plain HTTP only to this machine in development); short timeouts; a settings file with an on/off switch.
- `merge.rs`: combines local and cloud passages by rank (reciprocal rank fusion), never by raw score, and collapses duplicates, keeping the local copy.
- `commands.rs`: every question searches both at once. The local search never waits on the cloud, and a down, slow or signed-out cloud falls back to local results with a plain note. A whole local document is sent only when the question names that file (for example "summarise the Turnip engagement letter"); otherwise the best passages are sent. A cloud document is always summarised from its best passages.
- The privacy receipt counts local and cloud passages and says whether the question text went to the cloud index, which is true even when no AI model was used.
- UI: cloud passages carry a "Cloud" label and an **Open in Google Drive / Microsoft 365** link, and open as a single passage; settings has the sign-in and on/off controls; the header shows the cloud state.

Checked: unit tests for merging, the client (token refresh, PKCE, wrong callbacks, HTTPS rule, failures) and end-to-end retrieval, plus a live test against the real backend (`cargo test live_server -- --ignored` with `backend/scripts/dev-connect.sh sample` running).

Not done: the Bedrock credential broker (the app still uses the person's own AWS login to call Claude), whole-document fetch from the cloud index, and a real Cognito deployment to sign in against.

## 16. Managing connectors from the desktop app

Sources has two tabs: files on this device, and **Connectors**. The Connectors tab asks the server which connectors it offers, shows the firm's connected sources with their status, and lets a firm administrator connect, sync, reconnect or disconnect them. Connecting opens the provider's own sign-in page in the browser; the provider sends the administrator back to the server, which stores the connection, and the app notices it appear. The server enforces who may do what, so an ordinary member sees a plain "only a firm administrator" message rather than a broken button.

Safeguards in the app: the sign-in address the server returns is opened only if it is the Google or Microsoft sign-in page (a compromised server cannot send the browser anywhere else), connection IDs are checked before they go into a request address, and every request goes through the same signed-in path as search.

Reconnecting revives the existing connection (new token, status active, back on the sync list) rather than adding a duplicate.

## 17. How search works as built

**Cloud (Postgres full-text, `backend/src/db.rs`).** A question becomes an OR search over its content words: question filler ("tell", "me", "about", "what") is dropped, at most 16 words are kept, and neighbouring words are also joined ("zero dock" also searches "zerodock"). Text is stemmed, so "men's" and "mens" match. A file whose **name** matches ranks above one whose text matches, and a name match is shown by the file's first passage. No file contributes more than three passages. Files the signed-in person may not read are filtered out inside the same query. An earlier all-words (AND) search returned nothing for natural questions and was replaced.

**Local (SQLite FTS5, `src-tauri/src/db.rs`).** The same idea with prefix matching, so "tax" finds "taxes". Because a prefix can match unrelated words ("mens" begins the Spanish "mensaje"), a local passage must contain at least half of the question's content words (rounded up) when the question has two or more. File-name matches are handled separately and are not filtered this way.

**Merge.** The desktop asks the cloud and searches locally at the same time, then combines the two lists by rank (reciprocal rank fusion), never by raw score, and collapses duplicates keeping the local copy. If the cloud is down or signed out, results fall back to local only with a plain note.

**Answer.** Up to 8 passages (about 12,000 characters) go to Claude with a system prompt that treats source text as data, forbids invented facts and requires the single word ABSTAIN when the material does not support an answer. Every claim must cite a retrieved passage or it is dropped. A full local document is sent only when the question names that file. Nothing is sent to the model when retrieval finds nothing.

**Not built:** meaning-based (embedding) search, reranking, and spelling-variant matching beyond the joined-words rule. Questions whose wording shares no words with the file ("the letter about paying late") depend on the text containing related words.

## 18. Links to the original files

Each cloud result carries a link to open the original file. Google Drive links come from the file's `webViewLink`, or are built from the file ID for files stored before links were kept; Microsoft links come from the item's `webUrl`. The server keeps only `https` addresses. The desktop accepts a link only if it is a Google Drive, Google Docs,

SharePoint, OneDrive or 1drv.ms page with no embedded credentials, checks it again when it is opened, and opens it in the person's browser. The service then decides whether that person's own account may see the file; the app adds no access.

## 19. When connectors sync

| Event | Timing |
|---|---|
| Ordinary sync after the last one finishes | 5 minutes (changes only: Google changes feed, Microsoft delta) |
| Full listing to catch missed deletions and lost access | every 6 hours |
| After a failed sync | 15 minutes |
| Provider rate limit | as long as the provider asks, at least 30 seconds |
| Worker checks for due work | every 10 seconds |
| "Sync now" button | immediately |
| Work lease on a connection | 30 minutes |

A new or edited file is usually searchable within 5 to 6 minutes. The first sync of a large drive reads one file at a time (about one per second in testing), so a drive of tens of thousands of files can take many hours. A lease is not yet renewed during a very long sync, files are not yet processed in parallel, and no progress or time estimate is shown. Permissions are refreshed on each sync; removal of a person from a Google or Microsoft **group** is not resolved because group membership is not built.

## 20. Reliability findings from testing

- A NUL byte in text extracted from a file made the database reject the whole sync batch, silently stalling the index at the same file count every cycle. Text is now cleaned before storage, the server's own error message is logged, and a regression test covers it.
- A question containing "about" caused the app to send the whole top-ranked local file, unrelated to the question, which made the model answer ABSTAIN. A whole file is now sent only when the question names it.
- Loose prefix matching let two Spanish documentation files from a code folder displace relevant cloud results (section 17).
- Development builds log counts and file names (never document text) to a local file to diagnose empty answers.

## 21. Running cost per user (estimate)

Assumptions, all unverified: Sonnet 5.5 at $2 input and $10 output per million tokens; a normal answer sends about 3,500 input tokens and returns 700; a price of $49.99 per user per month; $1.75 card fees and about $2 storage and sync per user; no churn, tax or support time.

| User profile | Questions a month | Claude cost | Margin as built | With a document-size cap and a 500-answer limit |
|---|---|---|---|---|
| Light | 220 | $3.31 | $42.93 | $42.93 |
| Typical | 660 | $12.74 | $33.50 | $36.59 |
| Heavy | 1,320 | $32.47 | $13.77 | $33.94 |
| Extreme | 3,300 | $237.27 | -$191.03 | $31.29 |

Whole-document summaries are the cost risk: one can send up to about 375,000 tokens. Cost controls (a cap on document size and a fair-use limit) matter more than the hosting choice. Hosting options from one small server (about $35 a month) to a hardened AWS setup (about $350 a month) change the break-even by only about nine users once compliance costs of roughly $3,500 a month (an assumption covering SOC 2, a penetration test, Google's assessment, insurance and legal) are included; break-even is about 95 to 104 users at launch, or under 10 in a pilot without compliance spending.

## 22. Known gaps

Not built: OCR for handwriting and languages other than English (OCR itself is built, section 25); old .doc and .ppt files; reader isolation from network and file system (the deployment must provide it); the AWS Secrets Manager store for connector tokens; group-membership resolution; per-user rate limits; the container image and services to run it; IAM database login; embeddings and reranking; the Bedrock credential broker; a deployed Cognito; self-service sign-up and billing. Claude Sonnet 5.5 access on the AWS account returned an access error (HTTP 403), so development uses the direct Claude API in test mode, which has no zero data retention; do not use real client documents in that mode. Connector client secrets and API keys were pasted into chat during development and should be rotated before any real use.

## 23. Test status

Backend: 101 tests against a real Postgres (isolation, API, connectors, sync, file reader, sign-in). Desktop: 54 tests plus one live test run on demand. Interface: 11 tests. Code style and static checks (formatting, clippy, TypeScript, lint) are clean. The infrastructure code passes validation only.

## 24. Choosing the model and limiting use

**Everyday model and an "Advanced analysis" switch.** Questions are answered by Claude Haiku 5.5 by default. A person who wants a more thorough answer turns on **Advanced analysis** in the ask box, which uses Claude Sonnet 5.5 for that question. The switch carries a plain warning that it uses about twenty times as much of the person's monthly AI allowance per answer, and an answer that finds nothing offers "Try again with advanced analysis". The choice is the person's, so nothing is decided by guessing from the question's wording. The privacy receipt names the model that answered. Setting `PRIVATE_AI_CONNECTOR_WORKSPACE_MODEL` forces one model for everything (to compare them). List prices: Haiku 5.5 $0.10 / $0.50 per million tokens (five times that above 100,000 prompt tokens), Sonnet 5.5 $2 / $10, Opus 5.5 $4 / $20; a normal answer costs about $0.0007 on Haiku and $0.014 on Sonnet. Whether Haiku is accurate enough on a firm's documents has not been measured, and the Bedrock model name for Haiku is unverified because Bedrock access is not yet working.

**Monthly allowance per person.** After each answer the desktop reports the model and token counts to the firm's server, which prices them from its own table and adds them to an append-only ledger (counts and cost only). The default limit is $15 per person per calendar month (UTC); a firm administrator can change it for the whole firm, and the server lists each person's use for administrators. Before each question the desktop asks the server whether allowance remains; when it does not, the question is not sent to Claude, search still works, and the message says when the month resets. If the server cannot be asked, or the person is signed out or has the cloud index off, the question goes ahead. The limit is therefore a guard against surprise bills and not tamper-proof: a modified app, or one with the cloud index switched off, could skip it. Enforcing it properly needs the credential broker, so that Claude requests pass through the firm's server.

## 25. Reading scans and pictures (OCR)

Scanned PDFs and pictures (PNG, JPEG, TIFF, BMP, GIF, WebP) are read with Tesseract, so their contents are searchable and can be cited by page. The sync worker looks for Tesseract and Poppler on the server; without them everything behaves as before and scans are found by name only (the worker logs "OCR is off"). A PDF that has a text layer is never sent to OCR, because the text layer is exact. A PDF with no text has each page drawn by Poppler (at most 3,000 pixels on the long side, about 250 dots per inch for an A4 page) and read by Tesseract, one page at a time, and each passage is labelled "Page 3 (scanned)". A picture becomes one passage labelled "Scanned image". Pages and pictures with almost no readable text (blank pages, photographs) are left as name-only files, which the dashboard reports as "no text found, even after reading scans".

Safety and cost: both programs run as separate processes with an empty environment, an empty working folder, one thread, and hard limits on time (90 seconds a page), processor time, memory (on Linux) and written file size, and a file that hangs or crashes them is retried three times and then given up on like any unreadable file. Nothing is sent to any other service. Limits: at most 500 pages of one PDF are read; pictures under 5 KB (icons and logos) are skipped without downloading, and pictures over 40 MB are not read; only English is installed by default (`OCR_LANGUAGES` adds others once their language data is installed); handwriting and low-quality scans read poorly, and OCR text can contain mistakes, so answers built from scanned pages should be checked against the page. OCR is much slower than reading a text file, so the first sync of a drive full of scans takes proportionally longer. Reading files again with the new reader happens automatically after an upgrade.

## 26. Search by meaning and reranking

**Meaning vectors on the server.** Each stored passage gets a vector of 384 numbers from the small open-source model BGE small (`bge-small-en-v1.5`, MIT licence), run on the server with ONNX Runtime. The model is downloaded once (about 130 MB); document text is never sent to any other service to be embedded. A background loop in the sync worker gives vectors to passages that have none, a few at a time and alongside syncing, so a drive becomes searchable by meaning while it is still being read, and a model change re-embeds everything. Vectors are stored as plain number arrays with the model's name beside them (nothing is ever compared across models), so this works on any Postgres; with pgvector the comparison can move into an index without changing what is stored.

**Hybrid search.** A search runs two searches inside the same permission and firm filters: the existing word search, and a meaning search that finds the passages whose vector points closest to the question's (a question is embedded with the model's recommended prefix). Passages must be at least 0.5 similar to count, a bar chosen from measurements: with a real question a passage with no words in common scored 0.61 to 0.65, and unrelated passages 0.36 to 0.44 (`SEMANTIC_MIN_SIMILARITY` changes it). The two ranked lists are merged by reciprocal rank, never by raw score, with no more than three passages per file, so an exact name or code still wins and a paraphrase can still be found. If the model is unavailable, search matches words only.

**Measured on a real drive.** On 12 plain-language questions written without the files' wording (for example "the quiet weekend away" for a file named "Retreat of Silence"), against a 4,930-passage church and youth-ministry Drive, searching by words alone found the right file among the top 24 for 6 questions, and hybrid search found it for 10. It is not better for every question: where a question's words do match, the order inside the top 24 sometimes shifts a little, which is why the reranking pass below matters. This is a small hand-made test, not a benchmark. Adding meaning search costs about 0.3 seconds a question in a development build.

**Reranking.** When more passages are found than will be sent (24 found, 8 sent), the desktop asks the everyday Claude model (Haiku) to put the passages that really answer the question first. It is shown the question and 350 characters of each passage and returns a list of numbers; a reply that cannot be read leaves the merged order

alone, and passages it leaves out are kept after those it ranks, so reranking can only reorder, never drop. It costs about $0.0003 a question and counts toward the person's allowance. It sees local and cloud passages together, which is what lets one ranking cover both sources.

**Short documents are sent whole.** After ranking, the top two documents are sent in full when they are 12,000 characters or shorter, instead of as passages. A passage can hold a table's line items while the total sits in the next passage, which otherwise produces a confident, partial answer. A question names a document, and so gets it read whole up to the larger limits in section 24, only when it contains every word of the file's name, or all but one for names of four words or more; sharing most of a short name ("men's retreat" and "Men's Retreat Planning") is treated as a topic, not a file. If a whole-document answer comes back empty, the question is asked again from the ranked passages.

**Local files.** Meaning search is for the firm cloud index. Files on a person's own computer are still found by words, and reranking is how they are placed fairly beside cloud results. Giving the device its own model would let local files be searched by meaning too, at the cost of shipping a model with the app; that is not built.

**Costs and limits.** Embedding on a development Mac runs at about 5 passages a second (a drive of 5,000 passages takes about 15 minutes, once; larger batches used enough memory to get the process killed on a 16 GB machine under load); a production server with an optimised build and more processor cores is several times faster. The model needs a few hundred megabytes of memory in each of the API and the worker. The comparison reads every vector a person may see, which is fine for tens of thousands of passages; beyond that, pgvector's index is the next step.

## 27. Company knowledge (first phase)

**What it is.** The server reads each document in the firm cloud index once and records what it says about the business: what kind of document it is (contract, invoice, proposal, report, policy, procedure, meeting notes, correspondence, form, presentation, financial statement, schedule, resume, reference or other), the entities it names (people, organisations, projects, products and places, with a suggested role of customer, vendor, employee, partner or contractor), the facts, procedures, decisions and policies it states, and links between entities (works for, customer of, vendor of, manages, part of, owns, located in, related to). The same fixed categories serve every kind of business; nothing is specific to roofing, accounting or law. From the document summaries it drafts a company profile: what the business does, its services, and the terms its documents use.

**Nothing is invented.** The model must return, for every entity, fact and relationship, a short quote copied from the document. The server checks each quote against the document text (ignoring spacing, capitals and curly quotes) and throws away anything whose quote is not there, anything with an unknown category, and any relationship that does not connect two entities that were also extracted. In a live test, none of the model's items needed to be thrown away, but the check stays because a model can still paraphrase.

**People confirm it.** Everything starts as a suggestion. An administrator can confirm, correct (which also confirms), or reject a fact or entity, and confirm or edit the profile. A rejected item is not suggested again when the document is read again. Staff see only confirmed items.

**Permissions.** An item is shown to a person only if at least one document that supports it is one they may open, using the same rule as search; quotes are shown only from documents they may open. Review needs the same visibility, so an administrator can only review what a document they can open supports. The overview page (counts, document types, the profile) is drawn from every document, including ones some staff cannot open, so it is for administrators only. Each firm's data is separated by the same row-level security as everything else.

**Freshness.** Each fact remembers the version of the document its quote came from. When a document changes and is read again, its old evidence is replaced; a confirmed fact that came from it is marked "check again", a confirmed fact left with no evidence is kept and marked outdated, and an unconfirmed suggestion with no

evidence disappears. A confirmed fact not re-confirmed for 180 days is shown as stale. The reading instructions carry a version number, so improving them reads documents again.

**It is off until an administrator turns it on.** Reading documents sends their text (up to 24,000 characters of each) to Claude, so the Knowledge screen explains this, estimates the cost, and needs a tick before it turns on. It needs an `ANTHROPIC_API_KEY` on the server; in production the same Bedrock zero-retention requirement as answers applies, which is not yet met. The model is Claude Haiku 5.5 by default (`ANALYSIS_MODEL` changes it) with structured JSON output; a document costs about $0.001 to $0.002, so a drive of 1,000 documents costs roughly $1 to $2 once. A failed reading is retried three times and then left alone until the document changes. Files on people's own computers are never included.

**Not built yet (second phase):** a graph view of the relationships, detection of contradicting policies and facts, a browsable staff portal, using confirmed facts to answer questions, other languages, and roles other than administrator for reviewers. The relationships are already stored and extracted.

---

## 28. Shared AI agents (first slice, read-only)

An agent is a saved, versioned configuration that a firm administrator creates and offers to staff: a name and role, instructions, the tools it may use, a model (standard Haiku or advanced Sonnet), an effort level, who may use it, a monthly budget and how many tasks it may run at once. **One shared agent, many isolated tasks:** each job a person gives an agent is its own task with its own saved conversation, so two people using John never share a context.

**Server-side and durable.** Tasks run in the cloud service's background worker, not on anyone's computer. Starting a task writes it and a queue row (IDs and times only). The worker claims queue rows with a ten-minute lease renewed every step, up to four tasks at once, and saves the whole conversation after every step; if the worker dies the lease runs out and another picks the task up from where it stopped. Each agent's own limit on simultaneous tasks is enforced under a lock on the agent, and a task over the limit waits and tries again. A task takes at most 12 steps.

**A task has the permissions of the person who started it.** Their identity (user, groups, email) is saved with the task and used for every search and document read, so an agent can never read what that person could not open; a document they cannot open looks the same as one that does not exist. Tools are read-only: search documents (keyword and meaning), read a whole document, and look up confirmed company knowledge. Document text is treated as data, never as instructions.

**Who sees what.** Staff see the agents offered to them (not their instructions, budget or spending) and their own tasks, including the request, report and the documents used. Administrators see every agent, its instructions, spending and all tasks' status, cost and activity, but not another person's request or report. The activity history records what happened ("Searched documents (3 found)"), never document text.

**Cost control.** The work is recorded against the requester's monthly AI allowance, and stops if they run out. Each agent can have a monthly budget; reaching it refuses new tasks and stops running ones.

**Versions.** Changing the instructions, tools, model or effort saves a new version; a task keeps the version it started on. Going back saves a copy of the old version as the newest, so history only grows.

**Not built yet:** agents handing work to each other, human approval steps, spreadsheet and calculation tools, actions that change anything (sending, writing), shared agent memory, and evaluations. The agents John-style template is created by an administrator from the Agents screen; Alex, Taylor and Casey are shown there as planned only. Running agents needs `ANTHROPIC_API_KEY` on the server.

## 29. Keeping and deleting agent work

**Retention.** A finished task's report and the conversation behind it are kept for the firm's retention period: 90 days by default, chosen by an administrator from 30 days, 90 days, 180 days, 1 year, or "until their owners delete them". A background job deletes what is due. Deleting removes the conversation, the activity history, the request, the report, the edited version and the list of files used; what remains is the date and the cost, so budgets and allowances still add up. The deletion is recorded in the audit log as an ID.

**Asking first.** A week before a report is deleted, its owner (only the owner) is asked in a pop-up whether to keep it. "Keep it" keeps it until they delete it; "Let it be deleted" stops the asking and it is deleted on its date; "Ask me later" closes the box and it returns the next time the app opens. Owners can also delete a report at any time. Changing the firm's period never deletes anything without at least a week's notice.

**Reusing a conversation safely.** Each stored tool result records which files its text came from. Before a stored conversation is shown to the model again, text from any file the owner can no longer open is replaced with a notice.

**What others see.** Anyone other than the owner, administrators included, sees that a task happened, when, which agent and what it cost, with a neutral title and a history that says only "Used a tool". File names and the start of the request, which the title used to show, are private to the owner.

**Audit.** The worker now records agent task completions and failures (with the IDs of the files used), scheduled runs, skips and pauses, deletions, and cancellations, never any content.

**Production guard.** The server sends document text straight to the Claude API (no zero data retention) only when `ALLOW_INSECURE=1`, which is for local development. Otherwise agents and company knowledge stay off until the server can use Amazon Bedrock with zero data retention, which it cannot yet.

## 30. Dropbox and DocuSign connectors

Both are read-only and follow the same rules as Google Drive and Microsoft 365: files stay on the provider, the firm's index holds text and permissions, and a person finds only what the provider says they may open.

**Dropbox.** Reads the connecting account's files, shared folders and, for a business account, the team space. The first sync lists everything; later syncs follow Dropbox's change cursor, and a cursor Dropbox forgets triggers a full re-listing. Files are identified by lower-cased path because that is all Dropbox says about a deleted file, and a deleted folder (reported once) removes everything inside it. Permissions come from Dropbox's own sharing lists: the people and invited email addresses a file or its shared folder was shared with. A file that is not shared is readable only by the connecting account. Files shared only with a Dropbox group are not shown to the group's members, because Dropbox does not say who is in a group without a team-admin permission; this shows too little rather than too much. Public links are never access. Scopes: `files.metadata.read`, `files.content.read`, `sharing.read`, `account_info.read`, with offline access so the refresh token lasts.

**DocuSign.** Each envelope becomes two searchable items: its documents combined into one PDF, and a short text summary of its status (who sent it, who has signed, who is still outstanding), so "what is waiting for a signature?" can be answered. Readable by the sender and everyone the envelope was sent to, by email address, plus the connecting account. Later syncs list only envelopes whose status changed since the previous sync began (less a day, for safety); an unchanged envelope is recognised by its version and not downloaded again. DocuSign has no read-only scope for envelopes, only `signature`, which also allows sending; this server only ever reads, but the token itself is broader than we would like. DocuSign requires the client in a Basic authorization header and replaces refresh tokens as they are used, so the worker saves the replacement at the end of each sync. The API address DocuSign reports for the account is only accepted if it is one of DocuSign's own (`*.docusign.net`), since the access token is sent there.

**Setting them up.** Register an app with each provider and put its credentials in the server's environment (for local development, `backend/.env.local`): `DROPBOX_CLIENT_ID` and `DROPBOX_CLIENT_SECRET`; `DOCUSIGN_CLIENT_ID` (the integration key) and `DOCUSIGN_CLIENT_SECRET`, with `DOCUSIGN_ENVIRONMENT=production` for the real service (the default is DocuSign's developer sandbox). Each app must list the server's `OAUTH_REDIRECT_URI` as a redirect address. DocuSign requires a "go-live" review before an integration key works against real accounts.

**Not verified against the real services.** Both connectors were built from the providers' documentation and tested against stand-in servers that follow it. Details I could not confirm (for example whether a DocuSign list call covers other users' envelopes for an account administrator) need a first run against a real developer account.

## 31. Who a connected source is for: only me, or the entire workspace

Each connection has a scope, chosen before signing in to the provider.

- **Only me (private).** Only the person who connected it can find any of its files, whatever the provider says about sharing. Any employee may connect a source this way, and nobody else, an administrator included, can see, search, list, sync or remove it; to them it does not exist.
- **Entire workspace.** Everyone at the firm can search it, and each person finds only the files the provider says they may open (how connections have always worked). Only an administrator may connect for the whole workspace.

**How it is enforced.** Every search, document read, agent task and report decides who may see a file from one list stored with the file. A private connection records only its owner on every file it syncs, so every feature respects it with no special cases. The sync checks the setting at the moment it writes each file and holds it still until it has committed, so switching a connection to private cannot be missed by a sync in progress.

**Changing it later.** Only the person who connected a source can change its scope, and only an administrator can open it to the workspace. Going private takes effect at once, in the same moment as the change, across every file already stored. Going to the workspace leaves files owner-only until the sync, which starts immediately as a full re-listing, restores each file's real sharing; nothing is exposed before then.

**Company knowledge.** Files in a private connection are never read for company knowledge, and are left out of its counts, profile and lists. If a connection becomes private after its files were analysed, what was learned from them is removed: unconfirmed suggestions disappear, and confirmed facts are kept but marked outdated.

**Also fixed here.** The desktop app only recognised Google and Microsoft sign-in pages and document links, so connecting Dropbox or DocuSign, or opening their files, would have been refused. It now recognises theirs, and only theirs.

## 32. Emails and meetings prepared by agents (no mailbox or calendar access)

An agent can prepare an email or a meeting for a person. It can never send or book anything itself. By default nothing is requested from Google or Microsoft, so there is nothing for a company's administrator to approve; a person can send a prepared email from the app only if they connect their own Outlook for that (section 33).

**How it works.** An agent that has been given the "prepare emails" or "prepare meetings" tool proposes an item with the exact recipients, subject and message (or title, local times, invitees and notes). The proposal is stored for the person the task ran for and appears in their "Prepared for you" list. They read it, may edit any part, and choose where to open it: Gmail, Outlook or their mail app for an email; Google Calendar, Outlook Calendar or a calendar file for a meeting. Choosing one approves it and opens their own mail or calendar with everything filled in, and they press Send or Save themselves. A message too long for a link opens with the recipients and subject filled in and the text copied for pasting.

**Safeguards.**

- *Approval is the person's, only.* Nobody else, an administrator included, can see, approve or dismiss a proposal. Each is decided once.
- *The agent cannot reach outside.* Only the agent's worker can create a proposal; the service people use cannot. One task may prepare at most five. Text inside documents is never treated as a request: the agent is told to prepare something only when the person's request asks for it.
- *Addresses are strict.* Email addresses must be plain addresses, so nothing can add a header or a second recipient. Meetings need real times that run forwards for no more than 14 hours.
- *Look twice.* Recipients outside the person's own email domain are called out on the card.
- *The desktop app opens only what it should.* It opens only the compose and new-event pages of Gmail, Outlook and Google Calendar, or a plain `mailto:` link limited to recipients, cc, subject and body. A calendar file is made by the app, escaped so its text cannot add fields, checked, and written under a fixed name.
- *Audit and retention.* The audit log records that something was prepared, approved or dismissed, with its kind and task, never its text. Proposals are deleted with the report they came from.

**Later.** Real access to a mailbox or calendar (creating drafts or events directly, reading availability) needs per-person sign-in, administrator approval at most companies, and Google and Microsoft app verification. This feature is the approval step those would reuse.

## 33. Sending a prepared email from the app (Outlook)

A person can connect their own Outlook so that the draft an agent prepared can be sent from the app with one **Send** press, after reading it. It is the first thing this server does that leaves the firm, so it is opt-in at three levels and tightly limited.

**Connecting.** "Outlook email" is a connection of its own, for one person only (never the workspace), signed in with Microsoft's `Mail.Send` permission and no other mail permission: it can send as the person and cannot read, list or delete anything in the mailbox. It has no files and is never synced. The address it sends from is shown. The Microsoft app registration needs the delegated `Mail.Send` permission added; at companies that restrict app consent, an administrator has to approve it.

**Three switches, all required to send.** (1) An administrator turns on "let people send the emails their helpers prepare, from the app" for the firm: it is off until they do. (2) The person has connected their own Outlook. (3) The person presses Send on a specific draft.

**What is sent.** Exactly the text on the card, including any edits the person made, as plain text, saved in their own Sent Items. The server sends only an email the person has been shown: the agent cannot trigger a send, and the send call accepts only the person's own proposal.

**Safeguards.** Each email is sent at most once, even if Send is pressed twice at once; if Outlook refuses or cannot be reached, nothing counts as sent and the draft stays waiting. A lapsed sign-in marks the connection "needs reconnecting" instead of failing silently. At most 20 emails a day per person. If any recipient is outside the person's own email domain, they must tick that they have checked them first, and the server enforces this, not just the screen. Only the person the draft belongs to can send it: not another employee, not an administrator, whatever Outlook they have connected. Nothing in a document can cause a send, since sending needs a press of the button by the person.

**Audit.** The log records that an email was sent, how many recipients, whether it was edited and how many were outside the organisation, never its text or addresses.

**Not built.** Gmail (Google's send permissions are classed as sensitive or restricted and need app verification), calendar events created directly, and reading the mailbox.

**Figure 1. System architecture**

# Private AI Workspace: architecture

Where each part runs, what it does, and how far along it is. Your local files stay on your device. The firm cloud holds only text copied from sources the firm connects.

Status ● Built and tested ● Written, not deployed ● Not built yet

## YOUR DEVICE

**Desktop app**
Ask, search, Sources and Connectors screens. Shows which passages are local and which are cloud.

**Cloud client and merge**
Asks the firm cloud for matching passages, merges them with local results and picks the best.

**Local search index**
Encrypted database of your local files' text. Never uploaded anywhere.

**File reader (separate process)**
Reads PDFs, Word, Excel and scans on this device. Scans use the operating system's OCR.

**Your folders and files**
Stay where they are. The app only reads the folders you approve.

**Operating-system credential store**
Holds the index encryption key and the sign-in tokens (Keychain or Credential Manager).

**Ask Claude**
Sends your question and the chosen passages straight to Bedrock. Uses your own AWS login for now.

## FIRM CLOUD (your AWS account)

**API**
Search, connectors list, connect, sync, disconnect. Checks every request's sign-in and firm.

**Database (PostgreSQL)**
Synced text, file names and who may read each file. Firms are kept apart by row-level security inside the database. Planned on RDS.

**File reader (restricted process)**
Reads PDF, Word, Excel and PowerPoint files. Empty environment, time and memory limits.

**Sync worker**
Pulls files and permissions from Google and Microsoft every few minutes. Resumes after a crash.

**Sign-in (Cognito)**
Required two-step sign-in. The firm ID is set by us and users cannot change it.

**Secret store for tokens**
Holds each firm's long-lived Google and Microsoft access. Developer machines use a private file instead.

**Encryption keys (KMS)**
One key for the platform, one for audit logs, one per firm for files and secrets.

**File storage (S3)**
Created but not used yet. Original files are not kept.

**Audit and alerts**
CloudTrail records every file access, GuardDuty watches for threats, alerts go to email.

**Claude access broker**
Will give each app short-lived permission to call Claude directly, so firms need no AWS login of their own.

## OUTSIDE SERVICES

**Google and Microsoft sign-in**
Where an administrator approves read-only access. Passwords go only to them.

**Google Drive**
Read-only access to files and sharing settings.

**Microsoft 365**
OneDrive and SharePoint through Microsoft Graph. Read-only.

**Amazon Bedrock: Claude**
Answers questions. Zero data retention must be on before anything is sent.

### What the numbered arrows carry

1. Your question text goes to the firm cloud to be searched. Matching passages come back. Nothing from your local files goes up.
2. Sign-in to the firm's account. The firm and your role come from the sign-in token, never from the app.
3. The question and the best passages (local and cloud) go straight to Claude on Bedrock. The firm cloud never sees this step.
4. An administrator approves access at Google or Microsoft. A one-time code comes back to the API, which swaps it for the long-lived token.
5. The API reads and writes synced text and permissions. Every query is limited to one firm by the database itself.
6. The worker stores passages, names and permissions. It never stores the original files.
7. Each downloaded file is handed to a locked-down reader process and discarded; only extracted text comes back.
8. The worker fetches the firm's access token to read from Google and Microsoft.
9. Read-only file listing, sharing settings and downloads from the connected sources.
10. When a source is connected, the API saves its long-lived token in the secret store. The database never holds a token.

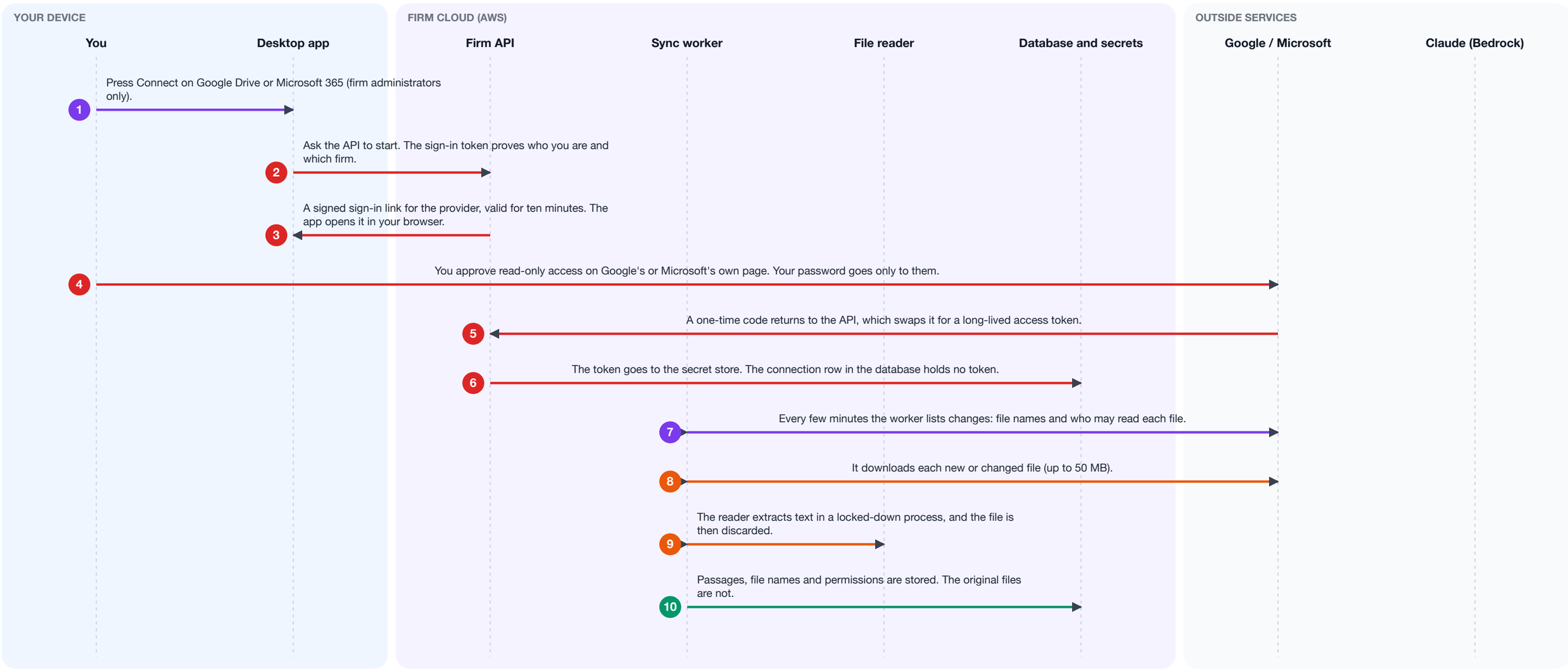
Status as of 2 November 2026. Nothing is deployed to AWS yet; the cloud side runs on a developer machine for testing.

**Figure 2. How a question and its data flow**

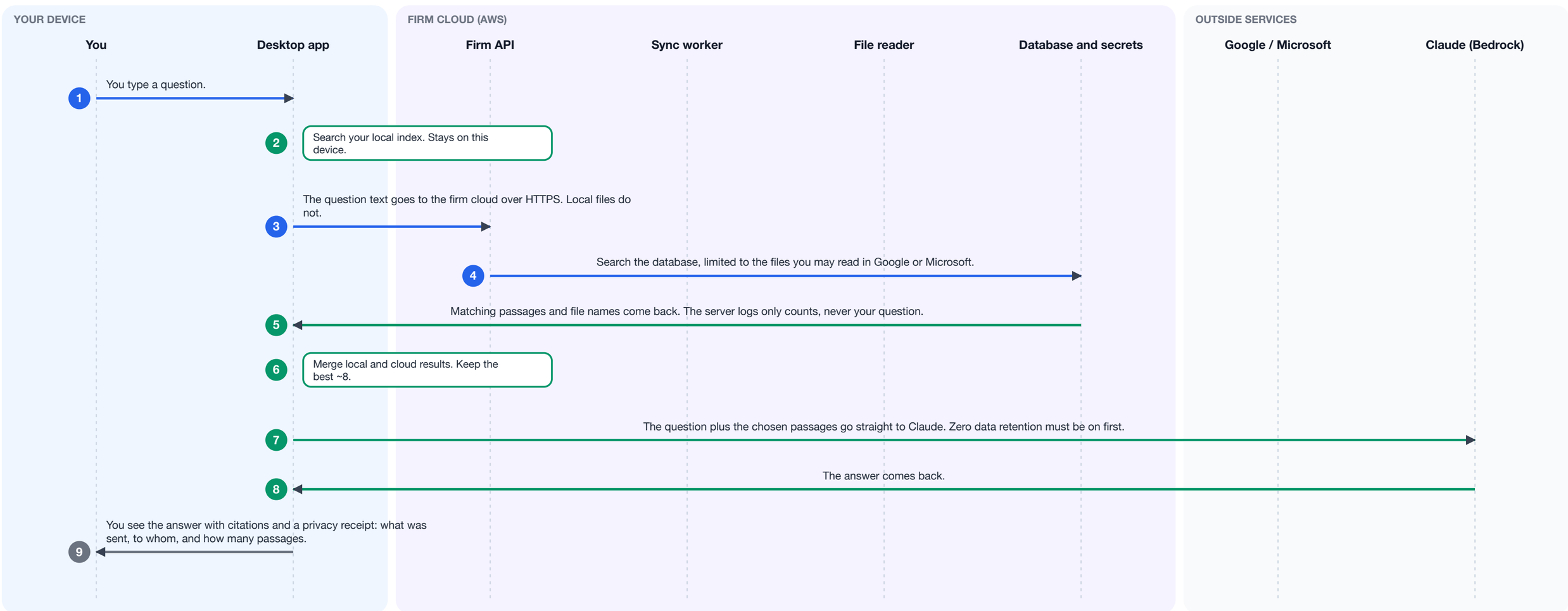
# Private AI Workspace: how your data moves

Two journeys: connecting a source and syncing it, then asking a question. Arrow colour shows what kind of data is moving. Local files never reach the firm cloud.

## A. Connecting a source and syncing its files




## B. Asking a question




**Kinds of data**

- Question text
- Sign-in and access tokens
- Passages of file text
- File names and who may read them
- File contents (passing through, not kept)
- Audit records (who, what, counts only)

Not shown: the firm cloud never calls Claude and never sees local passages. Today the app calls Claude with your own AWS login; a broker that hands out short-lived permission is planned. Audit records (who searched, how many results) are kept; question text is not.

Status as of 2 November 2026. Nothing is deployed to AWS yet; the cloud side runs on a developer machine for testing.

**Figure 3. What data lives where**

# Private AI Workspace: where your data lives

Every kind of user data, where it is kept, who can read it and how it is removed. Coloured dots show how far along each part is.

Status ● Built and tested ● Written, not deployed ● Not built yet

| Data | Where it lives | Who can read it, and how it is removed |
|---|---|---|
| ● **Your local files** | On your device, where you keep them | Not copied. Only the folders you approve are read. |
| ● **Text and index of your local files** | Encrypted database on your device. Key in the operating-system credential store. | Never uploaded to the firm cloud. Deleted from Settings > Clear local workspace. |
| ● **Text of connected files (Google, Microsoft)** | Firm cloud database, as plain text columns, in a shared database separated per firm by row-level security. Disk encryption with an AWS key. | Anyone with database access can read it. Deleted when the connector is removed; backups may keep it up to 14 days. |
| ● **Original connected files** | Not stored anywhere | Downloaded, read for text, then discarded. |
| ● **Who may read each connected file** | Firm cloud database, as email, domain and group identities | Refreshed with every sync and in full every six hours. Files whose permissions are unknown are hidden from everyone. |
| ● **Google and Microsoft access tokens** | Planned: AWS Secrets Manager. Developer machines: a private file readable only by you. | Read-only access. Deleted when the connector is removed. A person can also withdraw it at Google or Microsoft. |
| ● **Your sign-in** | Cognito (written, not deployed). The app keeps its tokens in the credential store. | Two-step sign-in required. Tokens expire and refresh. |
| ● **Your question text** | Typed in the app. Sent to the firm cloud to search it, and to Claude with the chosen passages. | The firm cloud does not store it (it logs only the length). The question history in the window is in memory and is lost when the app closes. |
| ● **Passages sent to Claude** | Amazon Bedrock | Sent only after zero data retention is confirmed on the AWS account. The firm cloud does not see this step. The direct Claude API test mode has no such check and is for test documents only. |
| ● **Privacy receipts** | Your device (encrypted database) | Each records what was sent, to whom, and how many passages came from local and cloud. |
| ● **Audit records** | Firm cloud database and AWS CloudTrail | Who searched or changed a connection, and counts. Never question or file text. |

Status as of 2 November 2026. Nothing is deployed to AWS yet; the cloud side runs on a developer machine for testing.